

CINEMATE

Application Web de Recommandation de Films

Dossier de Projet pour le Titre Professionnel Concepteur
Développeur d'Applications (CDA) — Niveau 6

Kagitheepan SRITHARAN

Année 2025 - 2026

DOSSIER DE PROJET

Titre Professionnel CDA — Concepteur Développeur d'Applications — Niveau 6

Kagitheepan SRITHARAN

Année 2025 – 2026

Cinemat — Dossier de Projet | Kagitheepan SRITHARAN | 2025-2026

Sommaire

1. [Tableau des Compétences — Titre Professionnel CDA Niveau 6](#)
 2. [Liste des Compétences Mises en Œuvre](#)
 3. [Cahier des Charges / Expression des Besoins](#)
 4. [Gestion de Projet](#)
 5. [Spécifications Fonctionnelles](#)
 6. [Maquettes et Enchaînement des Maquettes](#)
 7. [Modèle de Base de Données](#)
 8. [Script de Création de la Base de Données](#)
 9. [Diagrammes UML](#)
 10. [Spécifications Techniques](#)
 11. [Réalisations — Recommandation Personnalisée \(Fonctionnalité Principale\)](#)
 12. [Extraits de Code des Composants Métier](#)
 13. [Déploiement](#)
 14. [Plan de Tests](#)
 15. [Présentation des Éléments de Sécurité](#)
 16. [Veille Technologique — React & TypeScript](#)
- [Annexe A — Captures d'écran de l'application](#)
 - [Annexe B — Configuration Docker Compose](#)
 - [Annexe C — Pipeline CI/CD GitHub Actions](#)
 - [Annexe D — Liens utiles](#)
 - [Annexe E — Diagrammes UML](#)
-

1. Tableau des Compétences — Titre Professionnel

CDA Niveau 6

Référentiel Emploi Activités Compétences — Concepteur Développeur d'Applications — REAC
TP-01281 v04

Bloc	Compétence	Mise en œuvre dans Cinemate
AT1 — Développer une application sécurisée	C1 — Installer et configurer son environnement de travail	Docker Compose (5 services), Vite, Symfony 7.4 CLI
	C2 — Développer des interfaces utilisateur	React 19, TypeScript, Tailwind CSS 4, composants réactifs
	C3 — Développer des composants métier	Contrôleurs Symfony REST, algorithme de recommandation hybride
	C4 — Contribuer à la gestion d'un projet informatique	Git/GitHub, GitHub Actions CI/CD, documentation technique
AT2 — Concevoir et développer une application sécurisée organisée en couches	C5 — Analyser les besoins et maquetter une application	Cahier des charges, maquettes, diagrammes UML (PlantUML)
	C6 — Définir l'architecture logicielle d'une application	Architecture 3-tiers découplée, API REST, base hybride SQL/NoSQL
	C7 — Concevoir et mettre en place une base de données relationnelle	MySQL 8.0 (Doctrine ORM), MongoDB 6.0 (Doctrine ODM)
	C8 — Développer des composants d'accès aux données SQL et NoSQL	Repositories Doctrine, requêtes DBAL optimisées, DocumentManager MongoDB
AT3 — Préparer le déploiement d'une application sécurisée	C9 — Préparer et exécuter les plans de tests	Jest (14 tests frontend), PHPUnit (9 tests backend), GitHub Actions CI
	C10 — Préparer et exécuter le déploiement d'une application	Docker, docker-compose, pipeline CI/CD automatisé

Bloc	Compétence	Mise en œuvre dans Cinemate
	C11 — Planifier et mettre en œuvre une veille technologique	Veille React 19 / TypeScript, Hooks, Context API, Axios interceptors

2. Liste des Compétences Mises en Œuvre

2.1 Compétences techniques frontend

- Développement d'une SPA (Single Page Application) avec **React 19** et **TypeScript**
- Scaffolding et serveur de développement avec **Vite**
- Gestion du routing côté client avec **React Router DOM v7**
- Gestion de l'état global via **Context API** (AuthContext, MovieContext)
- Appels API REST asynchrones avec **Axios** et intercepteurs automatiques (JWT, consentement RGPD)
- Intégration d'un système de **Drag & Drop** interactif avec **@dnd-kit**
- Conception d'interfaces responsive et modernes avec **Tailwind CSS 4**
- Utilisation d'icônes SVG optimisées avec **Lucide React**

2.2 Compétences techniques backend

- Développement d'une **API REST** complète avec **Symfony 7.4** et **PHP 8.3**
- Authentification stateless via **JSON Web Token (JWT)** avec `lexik/jwt-authentication-bundle`
- Persistance relationnelle via **Doctrine ORM** (MySQL) avec migrations automatisées
- Persistance documentaire via **Doctrine ODM** (MongoDB) pour les logs et consentements
- Pipeline d'ingestion de données externes (API TMDb et OMDb) via **commandes Symfony Console**
- Hachage sécurisé des mots de passe avec **bcrypt** (algorithme `auto`)
- Système de notifications in-app et envoi d'**emails transactionnels** (Symfony Mailer)
- Gestion des en-têtes **CORS** pour les échanges cross-origin (`nelmio/cors-bundle`)

2.3 Compétences base de données

- Conception d'un modèle de données **hybride** (SQL pour les données relationnelles, NoSQL pour les logs)
- Création et gestion de tables MySQL avec clés étrangères, index uniques et colonnes JSON
- Mapping Object-Document MongoDB avec attributs PHP 8 natifs
- Migrations de schéma automatisées via `doctrine/migrations`

2.4 Compétences DevOps et déploiement

- Conteneurisation avec **Docker** (rédaction de Dockerfile pour PHP et Node)
- Orchestration de **5 conteneurs** avec **Docker Compose** (backend, frontend, MySQL, MongoDB, phpMyAdmin)
- Pipeline **CI/CD** automatisé avec **GitHub Actions** (lint, tests, build)
- Gestion de variables d'environnement pour les différents contextes (dev, test, prod)
- Versionnement du code avec **Git** et **GitHub**

2.5 Compétences conception et documentation

- Modélisation UML : diagrammes de cas d'utilisation, d'activité (PlantUML `.wsd`)
 - Rédaction de documentation technique détaillée
 - Rédaction d'un cahier des charges fonctionnel
-

3. Cahier des Charges / Expression des Besoins

3.1 Présentation du projet

Cinemat est une application web conçue pour répondre à un besoin courant : **choisir rapidement un film adapté au temps réellement disponible**. Beaucoup d'utilisateurs disposent de créneaux limités (pause, soirée courte, transports) et perdent du temps à parcourir des catalogues sans savoir si un film rentre dans leur planning.

L'application permet de **filtrer et recommander des films en fonction de la durée disponible**, tout en tenant compte de critères complémentaires (genre, humeur, popularité, plateformes de streaming). Elle vise à optimiser le temps de décision, améliorer l'expérience utilisateur et réduire la frustration liée au choix excessif sur les plateformes de streaming.

3.2 Identification du projet

Champ	Valeur
Nom du projet	Cinemat
Type	Application web full-stack
Auteur	Kagitheepan SRITHARAN
Formation	Titre Professionnel CDA — Niveau 6
Dépôt GitHub	https://github.com/Kagitheepan/cinemat-project
Statut	Fonctionnel en local (Docker)

3.3 Besoins fonctionnels

3.3.1 Authentification

- Inscription avec email, username et mot de passe (hashé bcrypt)
- Connexion et déconnexion sécurisées via JWT
- Persistance de la session via token Bearer dans les headers HTTP

3.3.2 Catalogue et exploration des films

- Importation automatisée du catalogue depuis l'API **TMDB** (films populaires, 7 dernières années, disponibles en streaming en France)
- Enrichissement des données (casting, réalisateur, bande-annonce, plateformes de streaming, durée via TMDB + fallback OMDb)
- Recherche par titre, acteur ou réalisateur avec filtres combinés (genre, durée)
- Affichage des détails complets d'un film (synopsis, casting, plateformes, bande-annonce YouTube)

3.3.3 Watchlist interactive

- Ajout / retrait de films dans une liste « À voir » et « Vus »
- Déplacement interactif entre les colonnes par **Drag & Drop** (`@dnd-kit`) ou boutons dédiés
- Boutons « Tout vider » par colonne avec confirmation
- Recherche et ajout de films depuis une modale intégrée

3.3.4 Recommandations personnalisées

- Moteur de scoring hybride basé sur : genres favoris du profil (+3 pts), genres implicites de la watchlist (+2 pts), plateformes communes (+5 pts), note TMDB (bonus)
- Exclusion automatique des films déjà présents dans la watchlist
- Filtres complémentaires sur la page de recommandations (titre, genre, durée)

3.3.5 Agenda de visionnage et rappels

- Planification de créneaux horaires pour regarder un film spécifique (calendrier interactif)
- Système de notifications in-app avec cloche dans la barre de navigation
- Envoi d'emails de rappel automatiques (24h avant) via commande Symfony Console planifiée

3.3.6 Découvrir un film (Roulette)

- Modale interactive permettant de filtrer par genre, durée maximum et nombre de propositions
- Animation « Machine à sous » avec flou progressif et révélation du résultat

3.3.7 Conformité RGPD

- Bandeau de consentement aux cookies (accepter / refuser)
- Persistance du consentement dans MongoDB avec version de politique

- Conditionnement des logs de connexion au consentement explicite de l'utilisateur
- Page « Politique de confidentialité » accessible publiquement

3.4 Besoins non fonctionnels

- **Sécurité** : JWT stateless, bcrypt, CORS configuré, vérification d'authentification sur toutes les routes sensibles
 - **Performance** : Cache serveur (fichier JSON + ETag HTTP 304), requêtes DBAL brutes pour le listing, OPcache PHP
 - **Disponibilité** : Environnement conteneurisé et reproductible via Docker
 - **Compatibilité** : Responsive, navigateurs modernes, Tailwind CSS pour l'adaptabilité mobile/desktop
-

4. Gestion de Projet

4.1 Planning et suivi

Phase	Tâches	Durée
Phase 1	Conception : cahier des charges, diagrammes UML, maquettes	2 semaines
Phase 2	Développement backend Symfony : API REST, auth JWT, CRUD, import TMDB	3 semaines
Phase 3	Développement frontend React : pages, composants, routing	3 semaines
Phase 4	Moteur de recommandation, RGPD, notifications, tests	2 semaines
Phase 5	Conteneurisation Docker, pipeline CI/CD GitHub Actions	1 semaine
Phase 6	Préparation titre CDA (dossier, diagrammes, révisions)	Novembre 2025 — Juin 2026

4.2 Environnement humain

Ce projet a été réalisé de manière individuelle. En tant qu'unique développeur, j'ai pris en charge l'intégralité du cycle de développement : conception, développement frontend et backend, intégration, tests et déploiement.

4.3 Environnement technique

Outil	Usage
Visual Studio Code	Éditeur de code principal
Git + GitHub	Versionnement et hébergement du code source
Docker Desktop	Conteneurisation et environnement local
phpMyAdmin	Interface graphique pour explorer et modifier la base de données MySQL
MongoDB Compass / Extension VS Code	Interface pour explorer la base MongoDB
GitHub Actions	Pipeline CI/CD automatisé (tests + build)
PlantUML	Création des diagrammes UML (fichiers <code>.wsd</code>)

4.4 Objectifs de qualité

- Code versionné avec des commits descriptifs et réguliers
 - Séparation claire frontend / backend (architecture découplée)
 - Sécurisation de toutes les routes sensibles côté serveur
 - Tests automatisés exécutés à chaque push via GitHub Actions
 - Documentation des choix techniques dans le dossier de projet
-

5. Spécifications Fonctionnelles

5.1 Architecture logicielle

L'application suit une architecture client-serveur découplée en trois tiers avec une base de données hybride :

- **Tier 1 — Présentation** : React 19 (SPA) + TypeScript + Tailwind CSS 4, servi par Vite (dev) ou Nginx (prod)
- **Tier 2 — Logique métier** : PHP 8.3 / Symfony 7.4 (API REST stateless)
- **Tier 3 — Données** :
 - **MySQL 8.0** (Doctrine ORM) : Utilisateurs, Films, Notifications
 - **MongoDB 6.0** (Doctrine ODM) : Logs de connexion, Consentements RGPD

5.2 Structure des dossiers

Dossier / Fichier	Description
frontend/src/pages/	Pages React (Home, Movies, MovieDetails, Watchlist, Calendar, Profile, Recommendations, PrivacyPolicy)
frontend/src/components/	Composants réutilisables (Navbar, AuthModal, DiscoverModal, CookieBanner, NotificationBell, MovieCard, TimeModal, AddEventModal)
frontend/src/context/AuthContext.tsx	Gestion globale de la session utilisateur et du token JWT
frontend/src/context/MovieContext.tsx	Gestion globale du catalogue de films et du cache de données
frontend/src/services/api.ts	Service Axios centralisé avec intercepteurs (JWT, consentement RGPD, gestion des 401)
backend/src/Controller/	Contrôleurs REST Symfony (Movie, Profile, Registration, Notification, Privacy, Auth)
backend/src/Entity/	Entités Doctrine ORM : User, Movie, Notification
backend/src/Document/	Documents Doctrine ODM : ConnectionLog, CookieConsent
backend/src/Command/	Commandes CLI : ImportMoviesCommand, SendRemindersCommand
backend/src/Service/	Services métier : TmdbService, OmdbService
backend/src/EventSubscriber/	Subscriber Symfony : LoginSubscriber (logs conditionnels RGPD)
docker-compose.yml	Configuration Docker Compose (5 conteneurs)
.github/workflows/ci.yml	Pipeline CI/CD GitHub Actions

5.3 Routes de l'application (Frontend)

Route	Page	Accès
/	Page d'accueil avec filtres de recherche et roulette	Public
/movies	Catalogue paginé avec filtres (genre, durée, recherche)	Public
/movie/:id	Détails d'un film (casting, plateformes, BA, recommandations similaires)	Public
/watchlist	Gestion de la liste « À voir » et « Vus » (Drag & Drop)	Connecté
/agenda	Calendrier de planification de visionnage	Connecté
/profile	Préférences utilisateur (genres, plateformes)	Connecté
/recommendations	Suggestions personnalisées basées sur le profil et la watchlist	Connecté
/privacy	Page de politique de confidentialité	Public

5.4 Endpoints de l'API (Backend)

Méthode	Route	Description	Auth
POST	/api/register	Inscription utilisateur	Non
POST	/api/login_check	Authentification JWT	Non
POST	/api/privacy/consent	Enregistrement du consentement RGPD	Non
GET	/api/movies	Liste des films (cache + ETag)	Non
GET	/api/movies/{id}	Détails complets d'un film	Non
GET	/api/movies/recommendations/for-you	Recommandations personnalisées	Oui
GET	/api/profile	Profil de l'utilisateur connecté	Oui
PUT/PATCH	/api/profile	Mise à jour du profil (watchlist, agenda, préférences)	Oui
GET	/api/notifications	Liste des notifications	Oui
PATCH	/api/notifications/read/{id}	Marquer une notification comme lue	Oui
PATCH	/api/notifications/read-all	Marquer toutes les notifications comme lues	Oui

6. Maquettes et Enchaînement des Maquettes

6.1 Parcours utilisateur non connecté

```
Page d'accueil (/) → Clic "Se connecter" → Modale d'authentification → Connexion →  
Accueil connecté  
Page d'accueil (/) → Clic "Inscription" → Formulaire d'inscription → Connexion →  
Accueil connecté  
Page d'accueil (/) → Catalogue (/movies) → Détails d'un film (/movie/:id)
```

6.2 Parcours utilisateur connecté

```
Accueil connecté → Catalogue → Fiche film → Ajouter à la Watchlist → Watchlist  
(/watchlist)  
Accueil connecté → Watchlist → Drag & Drop vers "Vus"  
Accueil connecté → Fiche film → Planifier → Agenda (/agenda)  
Accueil connecté → Recommandations (/recommendations) → Fiche film  
Accueil connecté → Profil (/profile) → Modifier genres / plateformes  
Accueil connecté → Bouton "Découvrir un film" → Modale Roulette → Fiche film  
Accueil connecté → Déconnexion → Accueil public
```

7. Modèle de Base de Données

7.1 Architecture Hybride SQL / NoSQL

Cinamate implémente une architecture de stockage hybride pour dissocier les transactions relationnelles structurées et les écritures volumétriques non structurées :

- **MySQL (Doctrine ORM)** : Données transactionnelles avec relations fortes (utilisateurs, films, notifications)
- **MongoDB (Doctrine ODM)** : Données d'audit et de conformité à forte écriture sans contraintes relationnelles (logs de connexion, consentements cookies)

7.2 Tables SQL (MySQL) — Description détaillée

Table : `user`

Stocke les comptes utilisateurs de l'application avec leurs préférences et données personnelles.

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique de l'utilisateur
username	VARCHAR(180)	NOT NULL, UNIQUE	Nom d'utilisateur unique
email	VARCHAR(255)	NOT NULL, UNIQUE	Adresse email (utilisée à l'inscription)
password	VARCHAR(255)	NOT NULL	Mot de passe hashé (bcrypt)
roles	JSON	NOT NULL	Rôles Symfony (ex: <code>["ROLE_USER"]</code>)
platforms	JSON	NOT NULL	Plateformes SVOD de l'utilisateur (ex: <code>["Netflix", "Disney+"]</code>)
favorite_genres	JSON	NOT NULL	Genres favoris (ex: <code>["Action", "Science-fiction"]</code>)
watchlist	JSON	NOT NULL	Structure <code>{toWatch: [...], watched: [...]}</code> avec les IDs films
agenda	JSON	NOT NULL	Événements de visionnage planifiés

Table : `movie`

Stocke les films importés depuis l'API TMDB. Chaque film est identifié par son `tmdb_id` unique.

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant interne unique
tmdb_id	INT	NOT NULL, UNIQUE	Identifiant TMDB du film
title	VARCHAR(255)	NOT NULL	Titre du film (en français)
description	TEXT	NULL	Synopsis du film
release_date	DATE	NULL	Date de sortie
poster	VARCHAR(255)	NULL	Chemin de l'affiche TMDB
backdrop	VARCHAR(255)	NULL	Chemin de l'image de fond TMDB
director	VARCHAR(255)	NULL	Nom du réalisateur
rating	FLOAT	NULL	Note moyenne TMDB (sur 10)
trailer_key	VARCHAR(255)	NULL	Clé YouTube de la bande-annonce (priorité FR > EN > Teaser)
genres	JSON	NOT NULL	Liste des genres (ex: ["Action", "Science-fiction"])
platforms	JSON	NOT NULL	Plateformes de streaming disponibles en France
cast	JSON	NOT NULL	Top 5 acteurs avec nom, rôle et photo
runtime	INT	NULL	Durée en minutes (TMDB + fallback OMDb)

Table : notification

Stocke les notifications de rappel envoyées aux utilisateurs pour les événements de leur agenda.

Colonne	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Identifiant unique
user_id	INT	FK → user(id), NOT NULL	Destinataire de la notification
message	VARCHAR(255)	NOT NULL	Contenu de la notification
type	VARCHAR(50)	NOT NULL	Type : 'reminder', 'info', 'alert'
is_read	BOOLEAN	DEFAULT false	Statut de lecture
email_sent	BOOLEAN	DEFAULT false	Email de rappel envoyé
movie_id	VARCHAR(255)	NULL	ID du film concerné
event_id	VARCHAR(255)	NULL	ID de l'événement d'agenda
created_at	DATETIME	NOT NULL	Date de création
event_date	DATETIME	NULL	Date de l'événement planifié

7.3 Collections MongoDB — Description détaillée

Collection : `connection_logs`

Enregistre l'historique de connexion des utilisateurs **uniquement si le consentement RGPD a été explicitement donné**.

Champ	Type BSON	Description
_id	ObjectId	Identifiant MongoDB auto-généré
username	string	Nom d'utilisateur qui s'est connecté
connectedAt	date	Date et heure de connexion

Collection : `cookie_consent`s

Conserve la trace formelle des choix de consentement cookies, répondant à l'obligation de preuve exigée par la CNIL / RGPD.

Champ	Type BSON	Description
<code>_id</code>	ObjectId	Identifiant MongoDB auto-généré
<code>consentId</code>	string	Identifiant unique du consentement (stocké dans un cookie HTTP)
<code>choice</code>	string	'accepted' ou 'refused'
<code>username</code>	string (nullable)	Nom d'utilisateur (si connecté au moment du choix)
<code>decidedAt</code>	date	Date et heure du choix
<code>policyVersion</code>	string	Version de la politique de confidentialité (ex: '2026-05-18')

8. Script de Création de la Base de Données

8.1 Migrations Doctrine (MySQL)

Le schéma MySQL est géré automatiquement par Doctrine ORM via le mécanisme de migrations. Le schéma est dérivé des attributs PHP 8 (`#[ORM\Column]` , `#[ORM\Entity]` , etc.) définis dans les entités.

Génération et exécution du schéma :

```
# Générer une migration à partir des changements dans les entités
docker exec cinemate-back php bin/console doctrine:migrations:diff

# Exécuter les migrations en attente
docker exec cinemate-back php bin/console doctrine:migrations:migrate

# OU forcer la mise à jour du schéma directement (développement)
docker exec cinemate-back php bin/console doctrine:schema:update --force
```

Exemple de migration — Ajout de la colonne `trailer_key` :

```
<?php
// backend/migrations/Version20260513074600.php

final class Version20260513074600 extends AbstractMigration
{
    public function up(Schema $schema): void
    {
        $this->addSql('ALTER TABLE movie ADD trailer_key VARCHAR(255) DEFAULT NULL');
    }

    public function down(Schema $schema): void
    {
        $this->addSql('ALTER TABLE movie DROP trailer_key');
    }
}
```


8.2 Schéma MongoDB (Doctrine ODM)

Le schéma MongoDB est défini par les attributs PHP 8 dans les classes `Document`. Les index sont créés via la commande :

```
docker exec cinemate-back php bin/console doctrine:mongodb:schema:update
```

Mapping du document `ConnectionLog` :

```
<?php
// backend/src/Document/ConnectionLog.php

#[MongoDB\Document(collection: "connection_logs")]
class ConnectionLog
{
    #[MongoDB\Id]
    private ?string $id = null;

    #[MongoDB\Field(type: "string")]
    private ?string $username = null;

    #[MongoDB\Field(type: "date")]
    private ?DateTimeInterface $connectedAt = null;
}
```

9. Diagrammes UML

L'ensemble des diagrammes UML est réalisé en **PlantUML** (fichiers `.wsd` à la racine du projet). Voir Annexe E.

9.1 Diagrammes de Cas d'Utilisation

Six diagrammes de cas d'utilisation ont été réalisés, un par module fonctionnel :

- **Authentification & Compte** (`uc_account.wsd`) : Inscription, connexion, déconnexion
- **Gestion des Films** (`uc_movie.wsd`) : Parcourir, rechercher, filtrer, voir les détails
- **Recherche avancée** (`uc_search.wsd`) : Recherche par titre/acteur/réalisateur, filtres par genre/durée
- **Gestion Personnelle** (`uc_personal.wsd`) : Watchlist (ajouter/retirer), Agenda, Préférences (genres/plateformes), Recommandations, Notifications
- **Backend / Import** (`uc_backend.wsd`) : Pipeline d'import TMDB, enrichissement OMDb
- **Packages** (`uc_packages.wsd`) : Vue d'ensemble des packages fonctionnels

9.2 Diagrammes d'Activité

Quatre diagrammes d'activité ont été réalisés pour les flux principaux :

- **Import automatisé TMDB** (`act.wsd`) : Flux complet de la commande `app:import-movies`
- **Ajout à la watchlist** (`act2.wsd`) : Parcours utilisateur pour ajouter un film
- **Système de recommandation** (`act3.wsd`) : Calcul du score et filtrage
- **Consentement RGPD** (`act4.wsd`) : Flux de gestion du bandeau de cookies

9.3 Diagramme de Classes

Le diagramme de classes est documenté dans `DIAGRAMME_CLASSES.md` et modélise la structure statique de l'application : entités `User`, `Movie`, `Notification`, documents `ConnectionLog`, `CookieConsent`, services `TmdbService`, `OmdbService`, et leurs relations.

10. Spécifications Techniques

10.1 Stack technologique

Couche	Technologie	Version	Ce que ça fait
Interface	React	19	Ce que l'utilisateur voit et avec quoi il interagit
Typage	TypeScript	5.x	Ajoute un typage statique au JavaScript pour plus de robustesse
Styling	Tailwind CSS	4	Framework utilitaire pour le design responsive
Outil de création	Vite	6.x	Crée le projet React et démarre le serveur de dev
Navigation	React Router DOM	7.x	Gère les pages sans recharger le navigateur
Drag & Drop	@dnd-kit	6.x	Permet le glisser-déposer interactif dans la Watchlist
Client HTTP	Axios	1.x	Communique avec l'API backend avec intercepteurs automatiques
Icônes	Lucide React	0.x	Fournit des icônes SVG optimisées et accessibles
Serveur	PHP	8.3	Reçoit les requêtes, vérifie les droits, répond en JSON
Framework	Symfony	7.4	Framework PHP structurant : routing, sécurité, injection de dépendances
ORM SQL	Doctrine ORM	3.x	Mappe les entités PHP aux tables MySQL

Couche	Technologie	Version	Ce que ça fait
ODM NoSQL	Doctrine ODM	2.x	Mappe les documents PHP aux collections MongoDB
Auth	LexikJWTAuthBundle	3.x	Gère l'authentification stateless via JSON Web Tokens
Base SQL	MySQL	8.0	Stocke les données relationnelles (utilisateurs, films, notifications)
Base NoSQL	MongoDB	6.0	Stocke les logs d'audit et les consentements RGPD
Films / Séries	TMDB API	v3	Fournit le catalogue de films (affiches, synopsis, casting, plateformes)
Durées (fallback)	OMDb API	v1	Fournit la durée des films si TMDB ne l'a pas
Conteneurisation	Docker + Compose	—	Orchestre l'environnement de développement reproductible
CI/CD	GitHub Actions	—	Exécute les tests et le build automatiquement à chaque push

10.2 Sécurité de l'application

La sécurité a été une priorité tout au long du développement. Voici les principales protections mises en place.

10.2.1 Mots de passe

Les mots de passe ne sont jamais stockés en clair. Ils sont hashés avec `bcrypt` via le composant `Symfony PasswordHasher` (`password_hash()` / `password_verify()`). Même si quelqu'un accède à la base, il ne peut pas lire les mots de passe.

10.2.2 Authentification stateless (JWT)

Contrairement aux sessions PHP classiques, Cinemate utilise des **JSON Web Tokens**. À chaque connexion, un token signé est généré par le serveur et renvoyé au frontend. Ce token

est ensuite envoyé dans l'en-tête `Authorization: Bearer <token>` de chaque requête. Le serveur vérifie la signature du token sans aucun état côté serveur (pas de session en mémoire).

10.2.3 Protection CORS

Les en-têtes CORS sont configurés via `nelmio/cors-bundle` pour n'autoriser que les origines légitimes du frontend. L'en-tête custom `x-consent-tracking` est explicitement autorisé pour le flux RGPD.

10.2.4 Contrôle d'accès

La configuration `security.yaml` de Symfony définit des règles d'accès précises :

- Routes publiques : `/api/register` , `/api/login` , `/api/movies` , `/api/privacy/consent`
 - Routes protégées : toutes les autres routes sous `/api` nécessitent `IS_AUTHENTICATED_FULLY`
-

11. Réalisations — Recommandation Personnalisée (Fonctionnalité Principale)

11.1 Description de la fonctionnalité

Le moteur de recommandation est la fonctionnalité centrale de Cinemate. Il analyse le profil de l'utilisateur (genres favoris, plateformes de streaming) et le contenu de sa watchlist pour calculer un **score de pertinence** pour chaque film du catalogue, puis affiche les résultats triés par score décroissant.

11.2 Algorithme de scoring

Le score est calculé selon la formule suivante :

$$\text{Score} = (\text{Gfav} \times 3) + (\text{Gimp} \times 2) + (\text{Pcomm} \times 5) + (\text{Note_TMDB} / 10)$$

- **Genres Favoris (Gfav)** : +3 points par genre du film correspondant aux genres favoris du profil
- **Genres Implicites (Gimp)** : +2 points par genre correspondant aux 3 genres les plus fréquents dans la watchlist
- **Plateformes Communes (Pcomm)** : +5 points si le film est disponible sur une plateforme déclarée par l'utilisateur
- **Note TMDB** : bonus proportionnel à la note (max +1 point)
- **Seuil** : seuls les films avec un score ≥ 2 sont affichés

11.3 Code backend — MovieController::getRecommendations

```
<?php
// backend/src/Controller/MovieController.php (extrait)

#[Route('/recommendations/for-you', name: 'api_movies_recommendations', methods:
['GET'])]
public function getRecommendations(EntityManagerInterface $entityManager):
JsonResponse
{
    $user = $this->getUser();
    if (!$user) {
        return $this->json(['message' => 'Non authentifié'], 401);
    }

    // 1. Profil de l'utilisateur
    $favoriteGenres = $user->getFavoriteGenres() ?? [];
    $platforms = $user->getPlatforms() ?? [];
    $rawWatchlist = $user->getWatchlist() ?? [];

    // Support du format { toWatch: [...], watched: [...] }
    $watchlistIds = array_merge($rawWatchlist['toWatch'] ?? [],
    $rawWatchlist['watched'] ?? []);

    // 2. Extraire les 3 genres dominants de la watchlist
    $repository = $entityManager->getRepository(Movie::class);
    $watchlistMovies = $repository->findBy(['id' => $watchlistIds]);

    $implicitGenresCount = [];
    foreach ($watchlistMovies as $wm) {
        foreach ($wm->getGenres() ?? [] as $genre) {
            $implicitGenresCount[$genre] = ($implicitGenresCount[$genre] ?? 0) + 1;
        }
    }
    arsort($implicitGenresCount);
    $implicitGenres = array_slice(array_keys($implicitGenresCount), 0, 3);

    // 3. Calculer le score pour chaque film du catalogue
    $allMovies = $repository->findAll();
    $scoredMovies = [];

    foreach ($allMovies as $movie) {
        if (in_array((string)$movie->getId(), $watchlistIds)) continue; // Exclure la
watchlist
```

```
$score = 0;
$movieGenres = $movie->getGenres() ?? [];

foreach ($favoriteGenres as $fg) {
    if (in_array($fg, $movieGenres)) $score += 3; // +3 par genre favori
}
foreach ($implicitGenres as $ig) {
    if (in_array($ig, $movieGenres)) $score += 2; // +2 par genre implicite
}
// +5 si plateforme commune
foreach ($platforms as $up) {
    foreach ($movie->getPlatforms() ?? [] as $mp) {
        if (stripos($mp, $up) !== false) { $score += 5; break 2; }
    }
}
if ($movie->getRating()) $score += ($movie->getRating() / 10); // Bonus note

if ($score >= 2) {
    $scoredMovies[] = ['movie' => $movie, 'score' => $score];
}
}

// Tri par score décroissant, retour des 30 meilleurs
usort($scoredMovies, fn($a, $b) => $b['score'] <=> $a['score']);
$top = array_slice($scoredMovies, 0, 30);
// ... sérialisation et retour JSON
}
```


11.4 Code frontend — Recommendations.tsx

```
// frontend/src/pages/Recommendations.tsx (extrait)

const Recommendations = () => {
  const { isAuthenticated } = useAuth();
  const [recommendedMovies, setRecommendedMovies] = useState<Movie[]>([]);

  useEffect(() => {
    const fetchRecommendations = async () => {
      if (!isAuthenticated) return;
      const response = await api.get('/movies/recommendations/for-you');
      setRecommendedMovies(response.data);
    };
    fetchRecommendations();
  }, [isAuthenticated]);

  // Filtres locaux supplémentaires (genre, durée, recherche textuelle)
  const filteredMovies = recommendedMovies.filter(movie => {
    const matchesDuration = maxDuration ? (movie.duration || 999) <= maxDuration : true;
    const matchesSearch = query ? movie.title.toLowerCase().includes(query) : true;
    const matchesGenre = selectedGenre ? (movie.genres || []).includes(selectedGenre) : true;
    return matchesDuration && matchesSearch && matchesGenre;
  });

  return <MovieGrid movies={filteredMovies} />;
};
```

12. Extraits de Code des Composants Métier

12.1 Pipeline d'import automatisé — ImportMoviesCommand

Ce composant métier est la commande Symfony qui alimente le catalogue de films. Il orchestre les appels vers l'API TMDB et OMDb, enrichit les données, et persiste les films en base MySQL.

```
<?php
// backend/src/Command/ImportMoviesCommand.php (extrait)

#[AsCommand(name: 'app:import-movies', description: 'Imports movies from TMDB API')]
class ImportMoviesCommand extends Command
{
    protected function execute(InputInterface $input, OutputInterface $output): int
    {
        // 1. Récupérer le mapping des genres (one-shot)
        $genreMap = $this->tmdbService->getGenres();

        for ($page = 1; $page <= $pages; $page++) {
            // 2. Récupérer les films disponibles en streaming en France (7 dernières
            années)
            $moviesData = $this->tmdbService->discoverStreamingMovies($page);

            foreach ($moviesData as $movieData) {
                $movie = $repository->findOneBy(['tmdbId' => $tmdbId]) ?? new
                Movie();
                $movie->setTmdbId($tmdbId)->setTitle($title)-
                >setDescription($overview);

                // 3. Résolution de la durée (TMDB → OMDb fallback)
                $runtime = $this->tmdbService->getMovieDetails($tmdbId)['runtime'] ??
                null;
                if (!$runtime || $runtime === 0) {
                    $runtime = $this->omdbService->getRuntimeByTitle($originalTitle,
                    $year);
                }
                $movie->setRuntime($runtime);

                // 4. Détection des plateformes de streaming (JustWatch via TMDB)
                $providers = $this->tmdbService->getWatchProviders($tmdbId);
                // ... extraction flatrate, ads, rent/buy pour la France

                // 5. Sélection ordonnée de la bande-annonce (FR > EN > Teaser)
                $videos = $this->tmdbService->getVideos($tmdbId);
                // ... priorité : frTrailer > enTrailer > anyTrailer > frTeaser > ...

                $this->entityManager->persist($movie);
            }
            $this->entityManager->flush(); // Batch flush par page
        }
        return Command::SUCCESS;
    }
}
```

```
}  
}
```

12.2 Consentement RGPD conditionnel — LoginSubscriber

Ce composant illustre l'architecture événementielle de Symfony. Le subscriber intercepte chaque authentification réussie et ne persiste le log de connexion dans MongoDB que si l'utilisateur a explicitement consenti au tracking.

```
<?php  
// backend/src/EventSubscriber/LoginSubscriber.php  
  
class LoginSubscriber implements EventSubscriberInterface  
{  
    public static function getSubscribedEvents(): array  
    {  
        return [Events::AUTHENTICATION_SUCCESS => 'onAuthenticationSuccess'];  
    }  
  
    public function onAuthenticationSuccess(AuthenticationSuccessEvent $event): void  
    {  
        $user = $event->getUser();  
  
        // Vérification du consentement RGPD via l'en-tête HTTP custom  
        $request = $this->requestStack->getCurrentRequest();  
        $hasConsent = $request && $request->headers->get('x-consent-tracking') ===  
'true';  
  
        // Ne pas logger si l'utilisateur n'a pas consenti  
        if (!$hasConsent) return;  
  
        $log = new ConnectionLog();  
        $log->setUsername($user->getUserIdentifier());  
        $log->setConnectedAt(new \DateTime());  
  
        $this->dm->persist($log);  
        $this->dm->flush();  
    }  
}
```

12.3 Intercepteur Axios — Injection automatique du consentement

Côté frontend, un intercepteur Axios injecte automatiquement le header de consentement dans chaque requête si l'utilisateur a accepté les cookies.

```
// frontend/src/services/api.ts

const api = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' },
});

// Intercepteur : injection du token JWT + consentement RGPD
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  // Le header x-consent-tracking est conditionné par le choix de l'utilisateur
  return config;
});
```

13. Déploiement

13.1 Environnement local — Docker Compose

L'environnement de développement local utilise Docker Compose pour orchestrer **5 conteneurs** interconnectés sur un réseau privé Docker.

13.1.1 Architecture Docker

Service	Image	Port exposé	Rôle
backend	php:8.4-cli (build)	8000:8000	API REST Symfony
frontend	node:20-alpine (build)	5173:5173	SPA React + Vite (dev server)
database	mysql:8.0	3306:3306	Base de données SQL
mongo-db	mongo:6.0	27018:27017	Base de données NoSQL
pma	phpmyadmin/phpmyadmin	8080:80	Interface graphique MySQL

13.1.2 Conteneur `cinemate-back` (Backend)

Le Dockerfile utilise l'image PHP 8.4 CLI avec installation des extensions PDO, MongoDB, Zip, Intl. Composer est copié depuis l'image officielle. Le serveur de développement intégré PHP écoute sur le port 8000.

```
FROM php:8.4-cli
RUN apt-get update && apt-get install -y git unzip libzip-dev libicu-dev libssl-dev \
    && docker-php-ext-install pdo pdo_mysql zip intl
RUN pecl install mongodb && docker-php-ext-enable mongodb
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
WORKDIR /app
EXPOSE 8000
CMD bash -c "composer install --no-interaction && php -S 0.0.0.0:8000 -t public"
```

13.1.3 Conteneur `cinemate-front` (Frontend)

Le Dockerfile utilise Node 20 Alpine pour installer les dépendances et lancer le serveur Vite en mode développement avec le flag `--host` pour permettre l'accès depuis l'hôte Windows.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host"]
```

13.1.4 Lancement de l'environnement local

```
# Cloner le projet
git clone https://github.com/Kagitheepan/cinemate-project.git
cd cinemate-project

# Lancer tous les conteneurs (build + start)
docker-compose up --build

# Accéder à l'application
# Frontend : http://localhost:5173
# Backend  : http://localhost:8000
# MySQL    : localhost:3306
# MongoDB  : localhost:27018
# phpMyAdmin : http://localhost:8080

# Importer les films dans la base de données
docker exec cinemate-back php bin/console doctrine:schema:update --force
docker exec cinemate-back php bin/console app:import-movies --pages=10

# Arrêter les conteneurs
docker-compose down
```

13.2 Intégration Continue — GitHub Actions

Le pipeline CI/CD est déclenché automatiquement à chaque push ou pull request sur la branche `main`. Il exécute deux jobs en parallèle :

Job 1 : Frontend

1. Checkout du code
2. Installation de Node.js 22 avec cache npm
3. `npm ci` (installation déterministe des dépendances)
4. `npm test -- --runInBand` (exécution des 14 tests Jest)
5. `npm run build` (vérification de la compilation TypeScript + Vite)

Job 2 : Backend

1. Checkout du code
 2. Installation de PHP 8.3 avec extensions (ctype, iconv, mbstring, mongodb)
 3. `composer validate` (validation du fichier composer.json)
 4. `composer install` (installation des dépendances)
 5. `php bin/phpunit` (exécution des 9 tests PHPUnit)
-

14. Plan de Tests

14.1 Configuration des tests

Environnement	Framework	Configuration
Frontend	Jest + @testing-library/react	jest.config.ts avec ts-jest , environnement jsdom , mocks des contextes
Backend	PHPUnit 9.6	phpunit.dist.xml , coût bcrypt réduit à 4 en mode test

14.2 Tests unitaires Frontend — AuthContext

Tests du hook `useAuth()` et de la gestion de session :

```
// frontend/src/context/AuthContext.test.tsx

describe('AuthContext - Connexion et Inscription', () => {
  it('devrait initialiser avec un utilisateur non connecté si aucun cookie valide',
  async () => {
    const { result } = renderHook(() => useAuth(), { wrapper });
    await waitFor(() => expect(result.current.isLoading).toBe(false));
    expect(result.current.user).toBeNull();
    expect(result.current.isAuthenticated).toBe(false);
  });

  it('devrait permettre la connexion et mettre à jour le state', async () => {
    // ... login puis vérification que isAuthenticated === true
  });

  it('devrait permettre la déconnexion et nettoyer le state', async () => {
    // ... logout puis vérification que user === null
  });

  it('devrait récupérer le profil au démarrage si le cookie est valide', async ()
  => {
    mockApi.get.mockResolvedValueOnce({ data: mockUser });
    // ... vérification que api.get('/profile') est appelé
  });

  it('devrait gérer les erreurs de récupération du profil', async () => {
    // ... vérification que l'erreur est gérée gracieusement
  });
});
```

14.3 Tests d'intégration Frontend — Home

Tests de la page d'accueil avec ses filtres de recherche :

```
// frontend/src/pages/Home.test.tsx

describe('Home - recherche et filtres', () => {
  it('filtre les films avec la recherche par titre', async () => {
    await user.type(screen.getByPlaceholderText('Titre, acteur, réalisateur...'),
    'matrix');
    expect(screen.getByRole('heading', { name: '1 résultat'
  })).toBeInTheDocument();
    expect(screen.getByRole('heading', { name: 'Matrix' })).toBeInTheDocument();
  });

  it('filtre les films par genre', async () => {
    await user.selectOptions(genreSelect, 'Action');
    expect(screen.getByRole('heading', { name: '2 résultats'
  })).toBeInTheDocument();
  });

  it('filtre les films selon le temps disponible', async () => {
    await user.selectOptions(durationSelect, '90');
    expect(screen.getByRole('heading', { name: '1 résultat'
  })).toBeInTheDocument();
  });

  it('applique le temps disponible de la modale de démarrage', async () => {
    // ... test de la modale "Combien de temps avez-vous ?"
  });
});
```

14.4 Tests d'intégration Frontend — MovieDetails

```
// frontend/src/pages/MovieDetails.test.tsx

it('affiche les informations principales, le casting et les recommandations', async
() => {
  expect(await screen.findByRole('heading', { name: 'Matrix'
})).toBeInTheDocument();
  expect(screen.getByText('Disponible en streaming sur')).toBeInTheDocument();
  expect(screen.getByText('Netflix')).toBeInTheDocument();
  expect(screen.getByText('Keanu Reeves')).toBeInTheDocument();
  expect(screen.getByText('Neo')).toBeInTheDocument();
  expect(screen.getByRole('heading', { name: 'Inception' })).toBeInTheDocument();
// Recommandation
});
```

14.5 Tests d'intégration Frontend — Watchlist

```
// frontend/src/pages/Watchlist.test.tsx

describe('Watchlist - Ajout et Gestion', () => {
  it('devrait afficher les films dans les bonnes catégories au chargement');
  it('devrait permettre d\'ajouter un film à la liste "à voir"');
  it('devrait permettre de déplacer un film de "à voir" vers "vus"');
  it('devrait permettre de retirer un film de la watchlist');
});
```

14.6 Tests unitaires Backend — MovieController

```
<?php
// backend/tests/Controller/MovieControllerTest.php

class MovieControllerTest extends TestCase
{
    public function testListReturnsCachedMoviesWithCacheHeaders(): void
    {
        // Vérifie que le cache fichier fonctionne et que les en-têtes HTTP sont
        // corrects
        // Cache-Control: public, max-age=60, stale-while-revalidate=300
        // ETag présent
    }

    public function testListReturnsNotModifiedWhenEtagMatchesCache(): void
    {
        // Vérifie que le serveur renvoie un HTTP 304 Not Modified
        // quand le client envoie If-None-Match avec le bon ETag
    }

    public function testSerializeMovieReturnsFullDetailPayload(): void
    {
        // Vérifie que le payload JSON contient tous les champs attendus :
        // id, tmdbId, title, year, releaseDate, rating, imageUrl, genres,
        // category, duration, description, backdropUrl, director, cast,
        // availableOn, trailerKey
    }
}
```

14.7 Résultats des tests

Tous les tests passent avec succès. Le pipeline CI/CD GitHub Actions affiche le rapport suivant :

Suite de tests	Nb de tests	Résultat
Frontend — AuthContext	5 tests	✓ PASS
Frontend — Home (filtres)	4 tests	✓ PASS
Frontend — MovieDetails	1 test	✓ PASS
Frontend — Watchlist	4 tests	✓ PASS
Backend — MovieController	3 tests	✓ PASS
Total	14 + 9 = 23 tests	✓ PASS

14.8 Tests de sécurité

ID	Scénario	Action testée	Résultat attendu	Statut
S01	Auth manquante	GET /api/profile sans token JWT	Erreur 401 Non authentifié	✓
S02	Token expiré	GET /api/profile avec token expiré	Erreur 401, localStorage nettoyé	✓
S03	RGPD — Refus	Connexion sans acceptation cookies	Aucun log dans MongoDB	✓
S04	RGPD — Acceptation	Connexion après acceptation cookies	Log présent dans connection_logs	✓
S05	Accès données tierce	GET /api/notifications avec JWT d'un autre user	Retourne uniquement ses propres notifications	✓
S06	Inscription doublon	POST /api/register avec email existant	Erreur 409 Email already used	✓

15. Présentation des Éléments de Sécurité

15.1 Synthèse des mesures de sécurité

Menace	Contre-mesure	Implémentation
Mot de passe compromis	Hachage bcrypt	<code>UserPasswordHasherInterface</code> (algorithme <code>auto</code>)
Session hijacking	JWT stateless	Token signé RSA (clé privée dans <code>config/jwt/</code>)
Accès non autorisé	Vérification JWT sur chaque requête	Firewall Symfony (<code>security.yaml</code> : <code>jwt: ~</code>)
CORS non sécurisé	Liste blanche des origines	<code>nelmio_cors.yaml</code> avec origines explicites
Vol de cookie consent	Cookie sécurisé	<code>Secure</code> , <code>HttpOnly</code> , <code>SameSite=Lax</code> , expiration 13 mois
Non-conformité RGPD	Consentement conditionnel	Header <code>x-consent-tracking</code> , <code>LoginSubscriber</code> , preuve dans MongoDB
Données sensibles en transit	HTTPS (production)	Certificat TLS obligatoire en production

15.2 Veille sur les vulnérabilités

15.2.1 Vulnérabilités surveillées

- **OWASP Top 10** : injection SQL (impossible avec Doctrine ORM/paramètres liés), XSS, CSRF, broken authentication
- **CVE PHP 8.3 / Symfony 7.4** : aucune vulnérabilité critique identifiée sur les versions utilisées
- **Dépendances npm React/Vite** : audit régulier via `npm audit`
- **Dépendances Composer** : audit régulier via `composer audit`

16. Veille Technologique — React & TypeScript

Dans le cadre du développement de Cinemate, une veille technologique a été effectuée sur les concepts clés de React et TypeScript utilisés dans le projet.

16.1 Hooks fondamentaux

useState — Gestion de l'état local

Le hook `useState` permet de créer et gérer une variable d'état dans un composant React. Lorsque la valeur change, le composant se réaffiche automatiquement.

```
const [searchQuery, setSearchQuery] = useState('');
const [recommendedMovies, setRecommendedMovies] = useState<Movie[]>([]);
const [isLoading, setIsLoading] = useState(true);
```

Dans Cinemate, `useState` est utilisé pour gérer l'état des formulaires, des listes de films, des résultats de recherche et des indicateurs de chargement.

useEffect — Effets de bord

Le hook `useEffect` permet d'exécuter du code après l'affichage du composant. Il est utilisé pour charger des données depuis l'API.

```
useEffect(() => {
  const fetchRecommendations = async () => {
    const response = await api.get('/movies/recommendations/for-you');
    setRecommendedMovies(response.data);
  };
  fetchRecommendations();
}, [isAuthenticated]); // Re-exécuté si isAuthenticated change
```

useContext — Consommation du contexte

Le hook `useContext` permet à un composant enfant d'accéder aux données partagées dans un Context.


```
const { user, isAuthenticated, login, logout, updateUser } = useAuth();  
const { movies, isLoading, getMovie, fetchMovieDetails } = useMovies();
```

16.2 Concepts fondamentaux

Context API — État global

Le Context permet de partager des données globales à travers toute l'application sans passer les props à chaque niveau.

```
// AuthContext.tsx  
const AuthContext = createContext<AuthContextType | undefined>(undefined);  
  
export const AuthProvider = ({ children }: { children: ReactNode }) => {  
  const [user, setUser] = useState<User | null>(null);  
  return (  
    <AuthContext.Provider value={{ user, isAuthenticated: !!user, login, logout,  
updateUser }}>  
      {children}  
    </AuthContext.Provider>  
  );  
};  
  
// Custom Hook pour simplifier l'accès  
export const useAuth = () => {  
  const context = useContext(AuthContext);  
  if (context === undefined) throw new Error('useAuth must be used within an  
AuthProvider');  
  return context;  
};
```

TypeScript — Typage statique

TypeScript ajoute un typage statique au code JavaScript, ce qui réduit les bugs à l'exécution et améliore l'autocomplétion dans l'IDE.

```
// Interfaces typées pour les données de l'application
export interface User {
  username: string;
  email: string;
  platforms: string[];
  favoriteGenres: string[];
  watchlist: string[];
  agenda: CalendarEvent[];
}

export interface Movie {
  id: string;
  title: string;
  genres: string[];
  duration: number | null;
  rating: number;
  // ...
}
```

Axios Interceptors — Logique transversale

Les intercepteurs Axios permettent d'injecter automatiquement des en-têtes dans chaque requête (JWT, consentement) et de gérer les erreurs (401 → déconnexion automatique).

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
    }
    return Promise.reject(error);
  }
);
```

16.3 Application dans Cinemate

Concept	À quoi ça sert	Où dans Cinemate
<code>useState</code>	Retenir une information qui peut changer	Filtres, listes de films, formulaires, état de chargement
<code>useEffect</code>	Charger des données au montage d'une page	Chaque page charge ses données via l'API Symfony
<code>useContext</code>	Partager des infos entre toutes les pages	<code>useAuth()</code> : utilisateur connecté. <code>useMovies()</code> : catalogue
Context API	« Tiroir global » accessible partout	AuthContext (session JWT) et MovieContext (catalogue)
React Router	Navigation entre pages sans rechargement	Routes déclarées dans <code>App.tsx</code> avec <code><Routes></code>
TypeScript	Typage statique pour la robustesse	Toutes les interfaces (<code>User</code> , <code>Movie</code> , <code>CalendarEvent</code>)
Axios Interceptors	Injection automatique JWT + gestion 401	<code>api.ts</code> : token Bearer + déconnexion automatique
<code>@dnd-kit</code>	Drag & Drop interactif	Watchlist : déplacement entre « À voir » et « Vus »
Custom Hooks	Réutilisation de logique métier	<code>useAuth()</code> , <code>useMovies()</code> encapsulent l'accès aux contextes

ANNEXES

Annexe A — Captures d'écran de l'application

 **À INSÉRER** : Captures d'écran de chaque page de l'application :

- Page d'accueil avec les filtres de recherche et le bouton « Découvrir un film »
 - Catalogue des films avec filtres par genre et durée
 - Page de détails d'un film (casting, plateformes, bande-annonce)
 - Page Watchlist avec Drag & Drop entre « À voir » et « Vus »
 - Page Agenda / Calendrier de visionnage
 - Page de recommandations personnalisées
 - Page de profil utilisateur (genres, plateformes)
 - Modale de découverte (roulette) en action
 - Bandeau RGPD (accepter / refuser)
 - Cloche de notifications avec le compteur
-

Annexe B — Configuration Docker Compose

Le fichier `docker-compose.yml` orchestre 5 services pour l'environnement local :

- **cinemate-back** : PHP 8.4 CLI avec extensions PDO, MongoDB, Zip, Intl + Composer + serveur PHP intégré
- **cinemate-front** : Node 20 Alpine avec Vite en mode dev (hot-reload via `--host`)
- **cinemate-mysql** : MySQL 8.0 avec volume persistant
- **cinemate-mongo** : MongoDB 6.0 avec authentification root et volume persistant
- **cinemate-pma** : phpMyAdmin pour l'exploration graphique de MySQL

Variables d'environnement injectées au backend :

Variable	Valeur
DATABASE_URL	mysql://root:root@database:3306/cinemat_users? serverVersion=8.0
MONGODB_URI	mongodb://root:root@mongo-db:27017
MONGODB_DB	cinemat_movies
JWT_COOKIE_SECURE	false (développement local)
OMDB_API_KEY	9713dcac

Annexe C — Pipeline CI/CD GitHub Actions

Le fichier `.github/workflows/ci.yml` définit le pipeline CI exécuté à chaque push sur `main` :

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm test -- --runInBand
      - run: npm run build

  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3', extensions: 'ctype, iconv, mbstring, mongodb' }
      - run: composer validate --no-check-publish
      - run: composer install --prefer-dist --no-progress --no-interaction --no-scripts
      - run: php bin/phpunit -c phpunit.dist.xml
```

Annexe D — Liens utiles

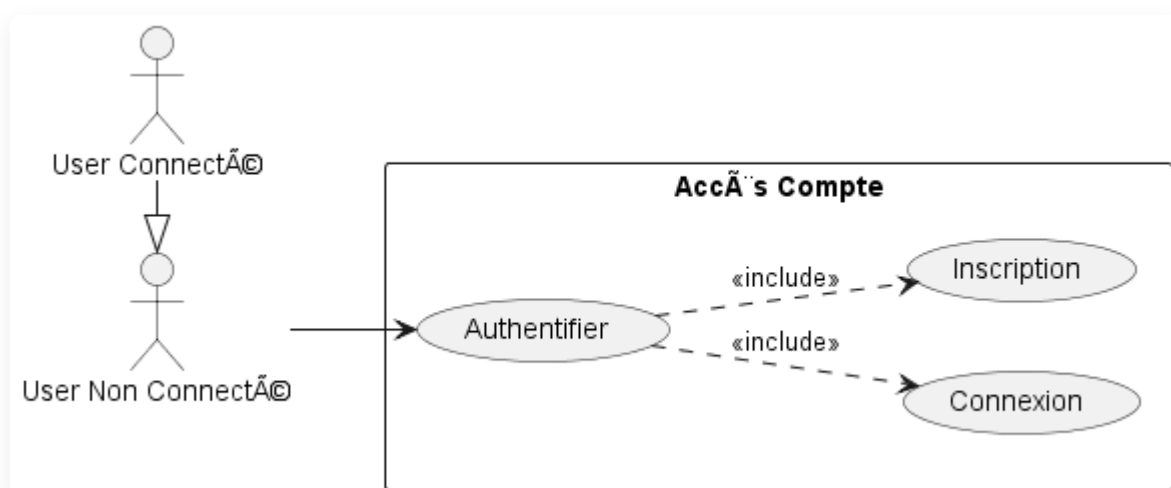
Ressource	URL
Code source GitHub	https://github.com/Kagitheepan/cinamate-project
API TMDB	https://developers.themoviedb.org/3
API OMDb	https://www.omdbapi.com
Documentation React	https://react.dev
Documentation Symfony	https://symfony.com/doc/current
Documentation Doctrine ORM	https://www.doctrine-project.org/projects/orm.html
Documentation Doctrine ODM	https://www.doctrine-project.org/projects/mongodb-odm.html
LexikJWTAuthenticationBundle	https://github.com/lexik/LexikJWTAuthenticationBundle

Annexe E — Diagrammes UML

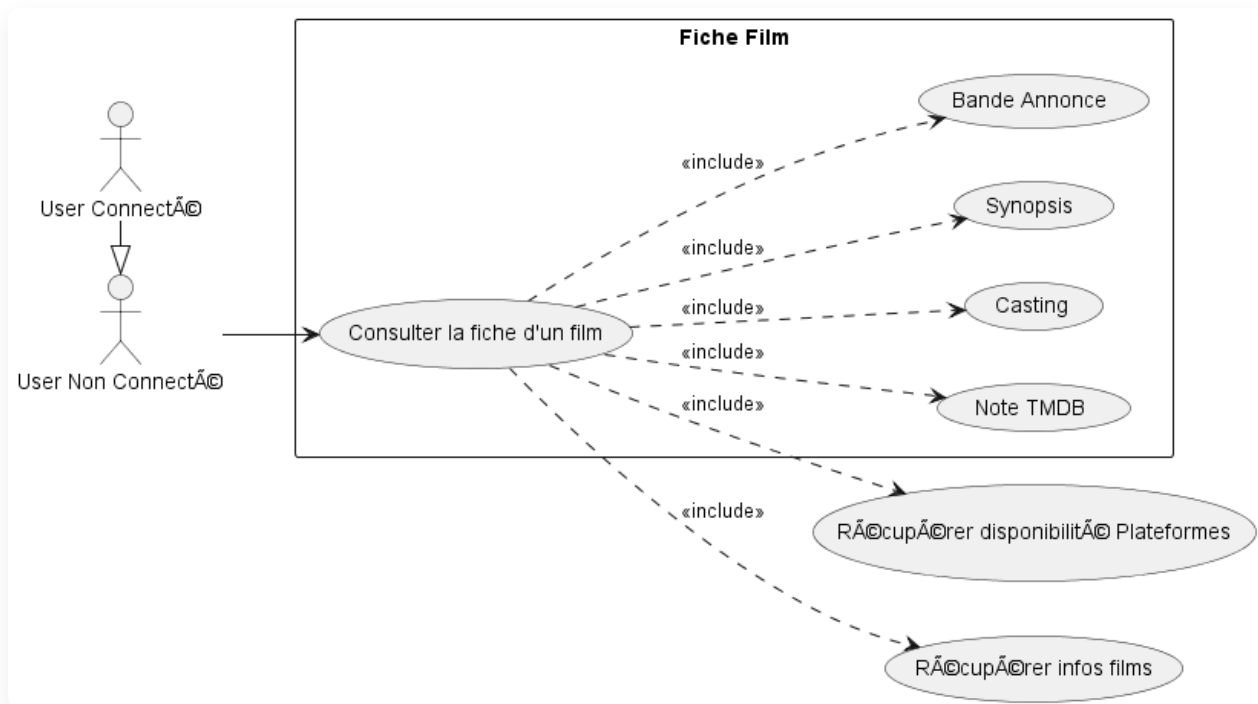
Les diagrammes de conception de l'application Cinemate ont été modélisés à l'aide de PlantUML. Leurs codes sources sont stockés sous forme de fichiers `.wsd` à la racine du projet et compilés sous forme d'images PNG.

E.1 Diagrammes de Cas d'Utilisation (Use Cases)

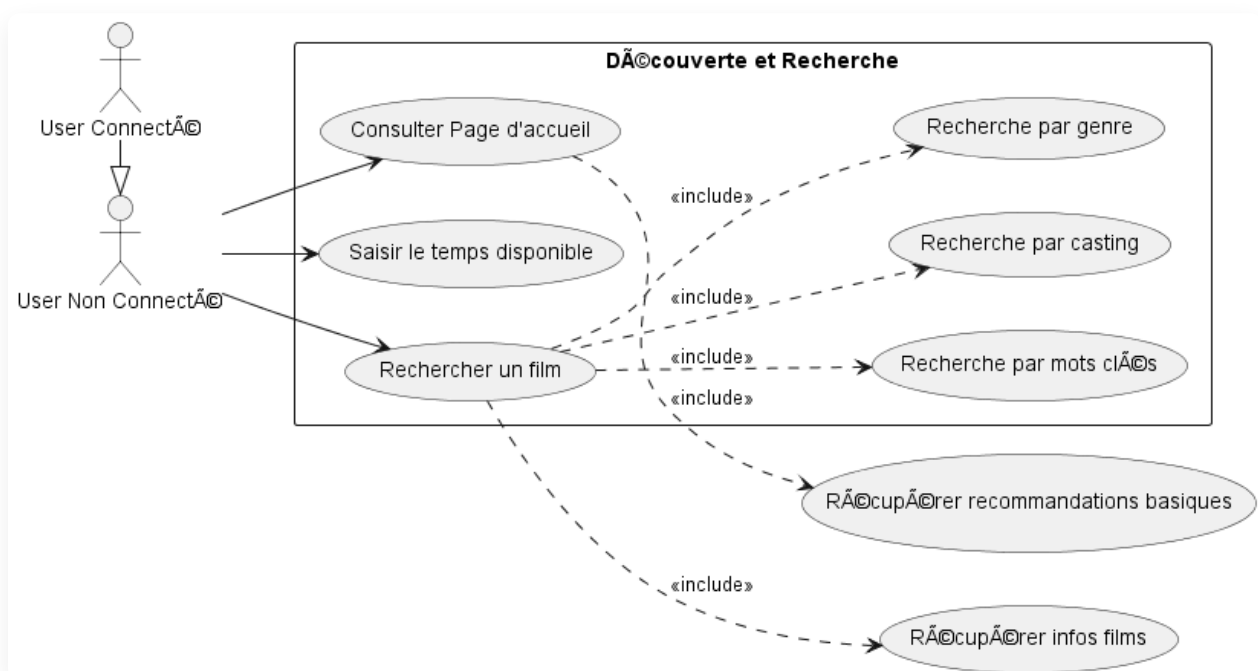
E.1.1 Authentification et Gestion de Compte (`uc_account.wsd`)



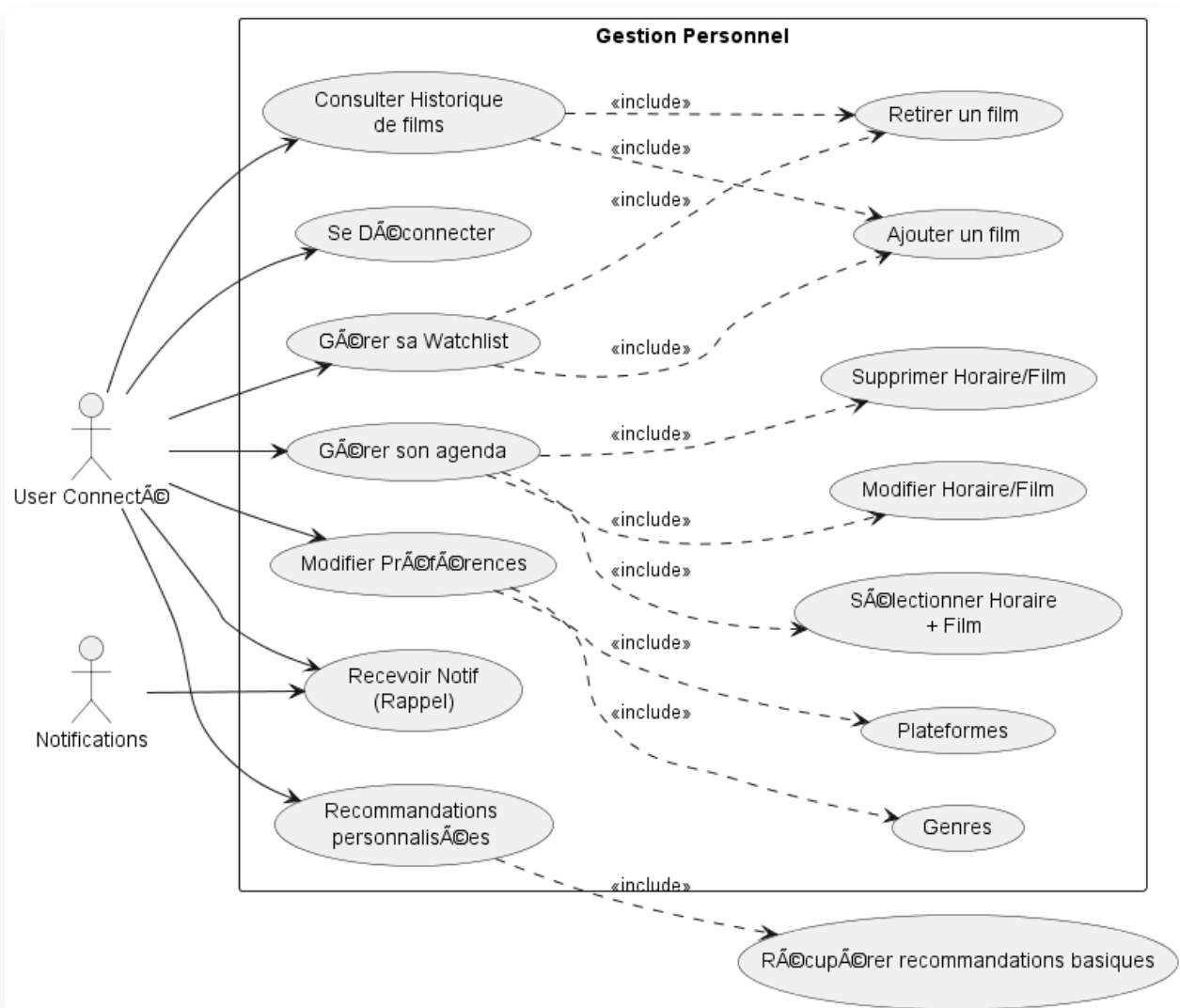
E.1.2 Consultation et Gestion des Films (uc_movie.wsd)



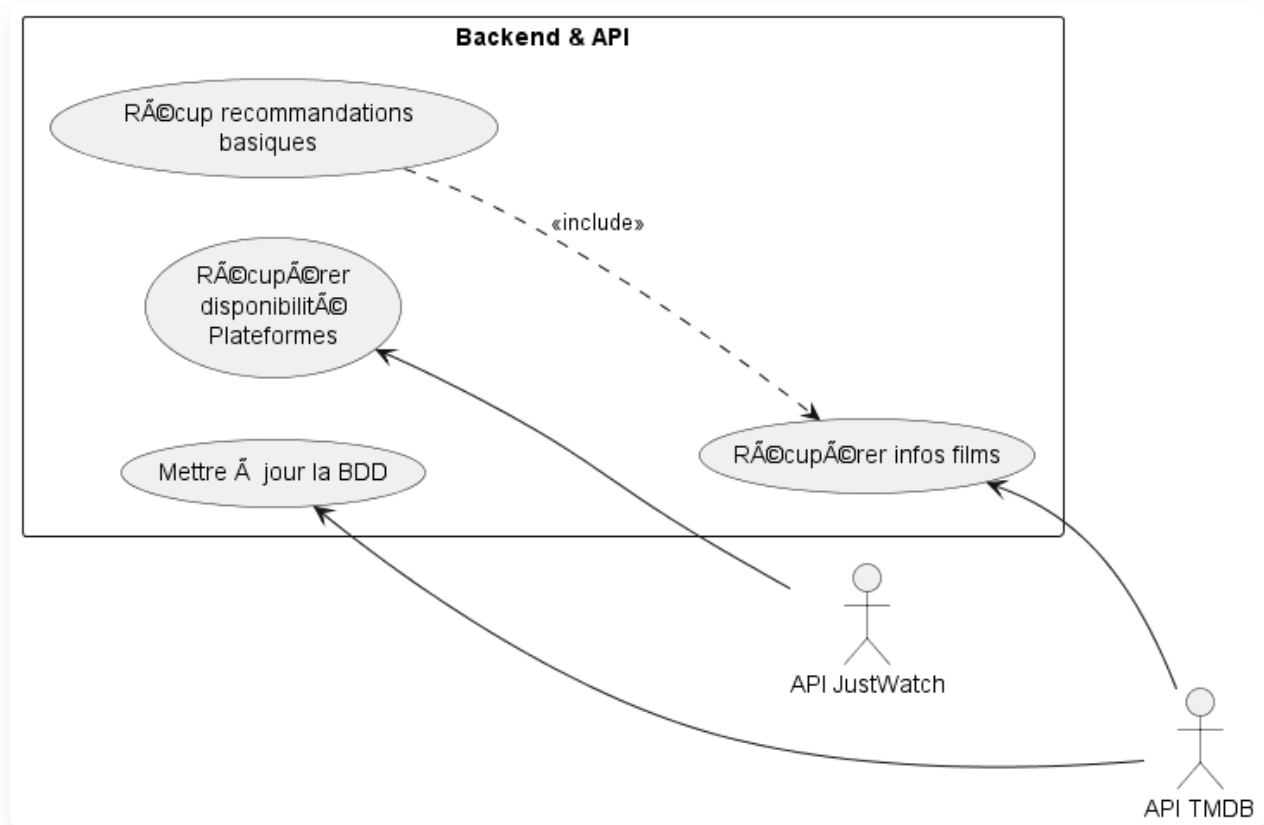
E.1.3 Recherche Avancée et Recherche Multi-critères (uc_search.wsd)



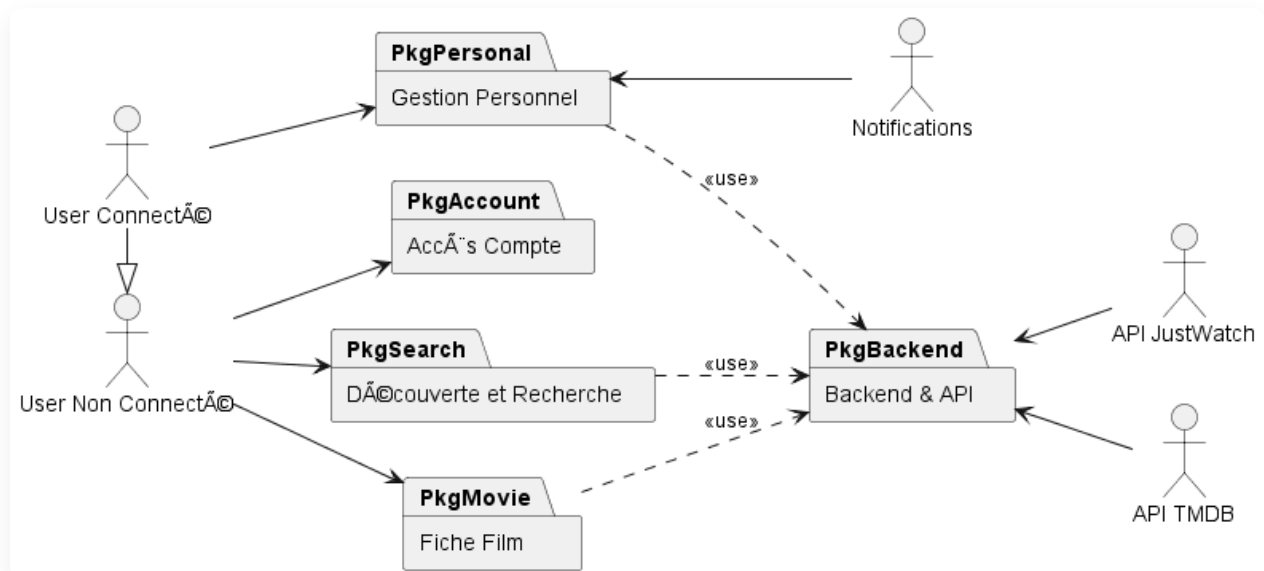
E.1.4 Espace Personnel, Watchlist, Agenda et Recommandations (uc_personal.wsd)



E.1.5 Ingestion de Données et Administration Backend (uc_backend.wsd)

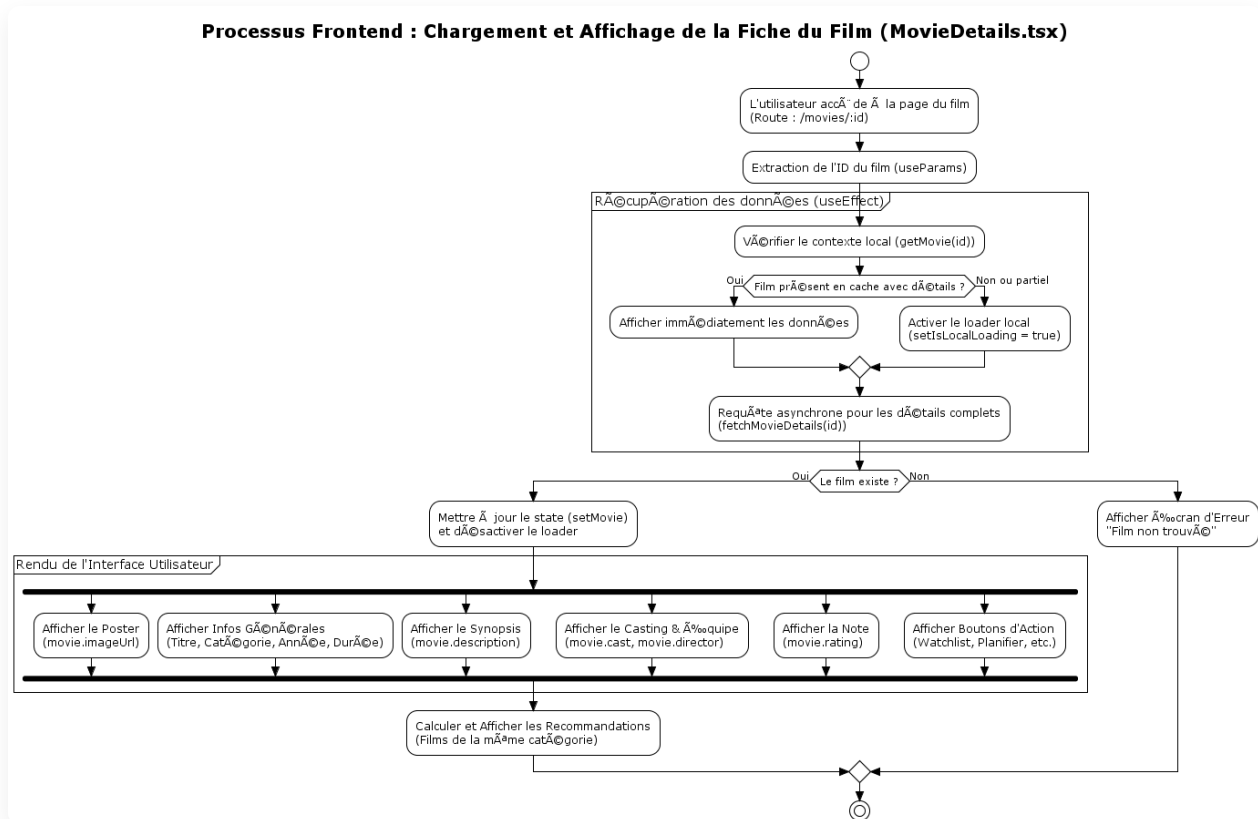


E.1.6 Vue d'Ensemble des Packages de l'Application (uc_packages.wsd)

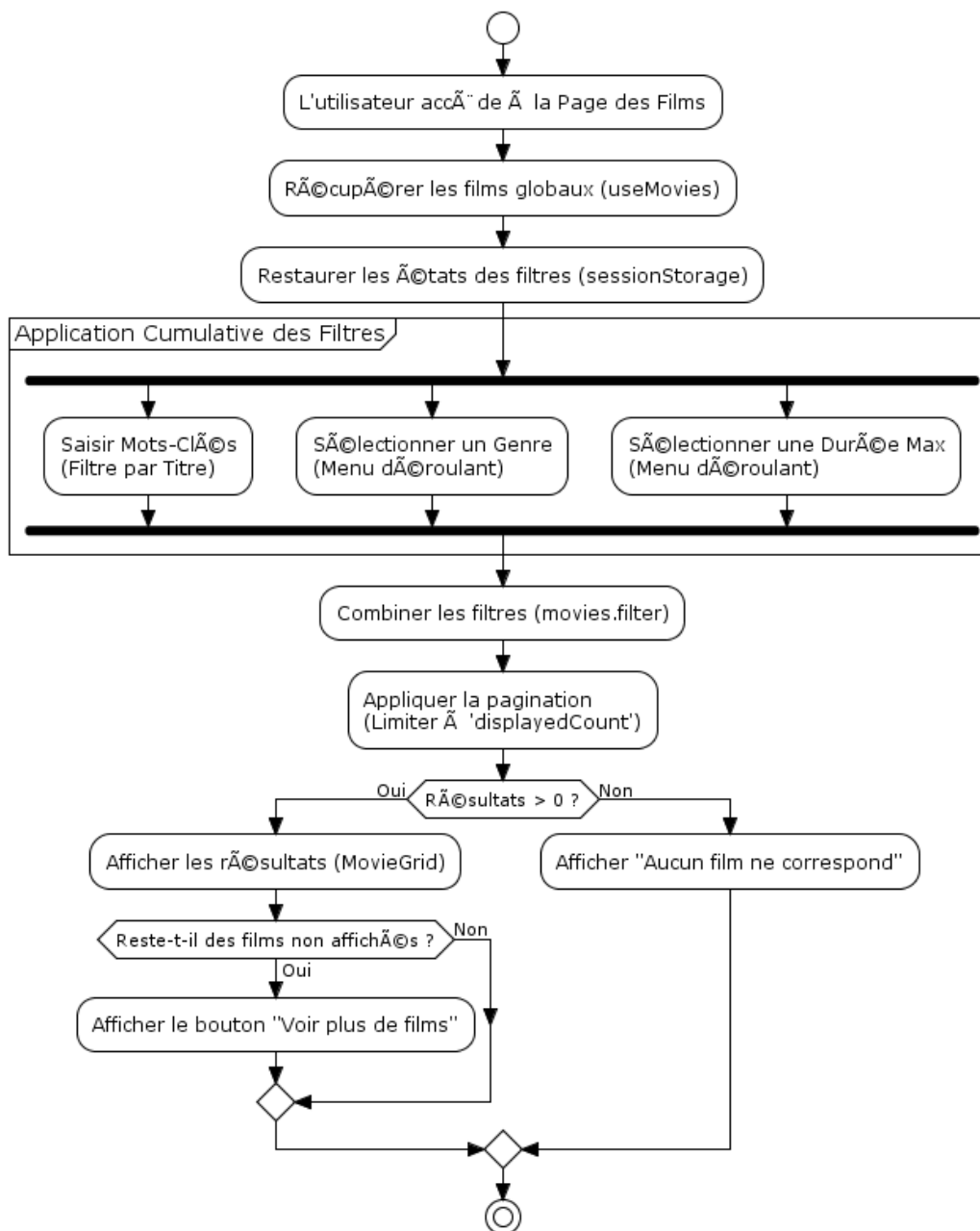


E.2 Diagrammes d'Activité (Workflow Processes)

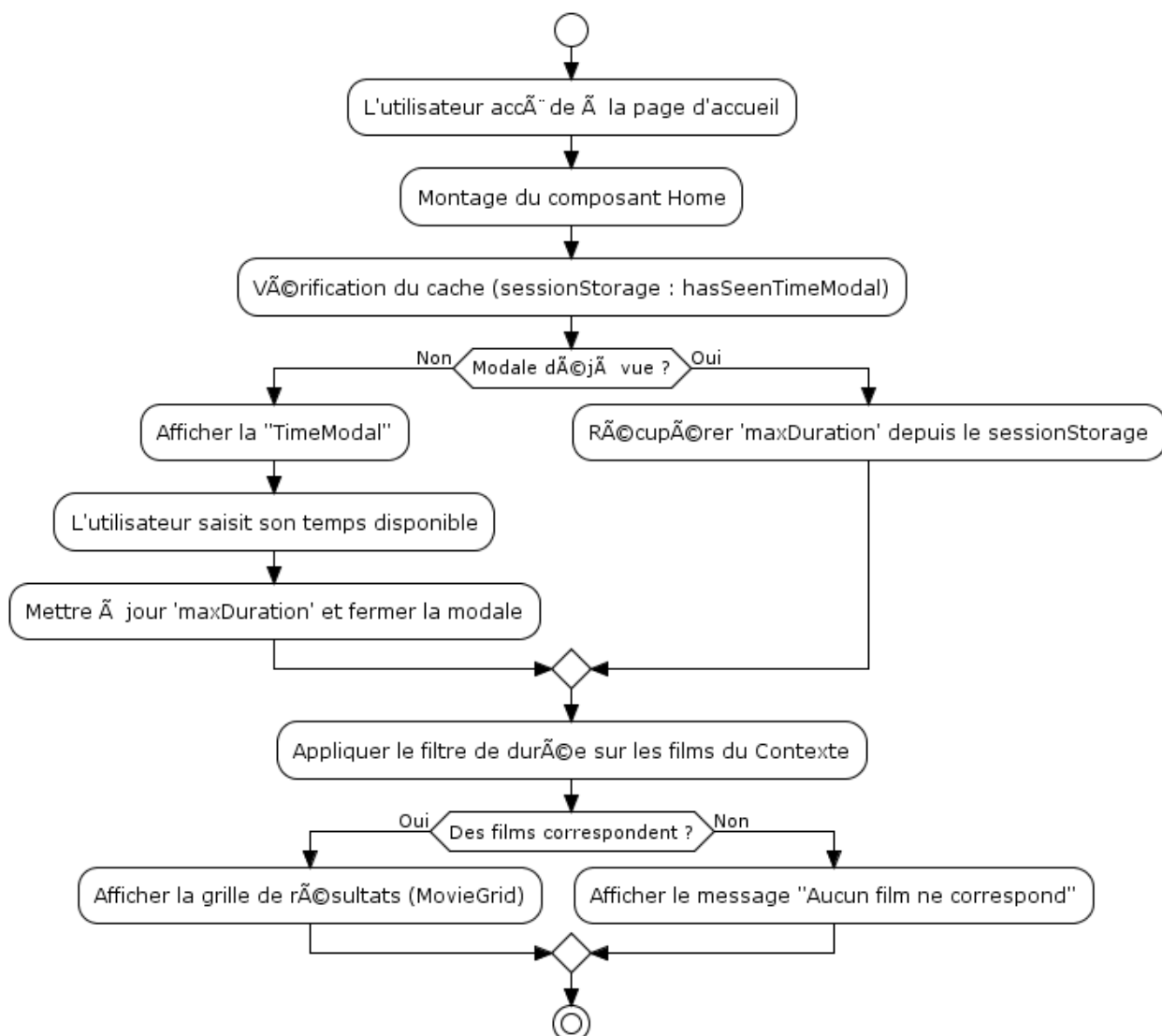
E.2.1 Processus de Chargement de la Fiche du Film (act.wsd)



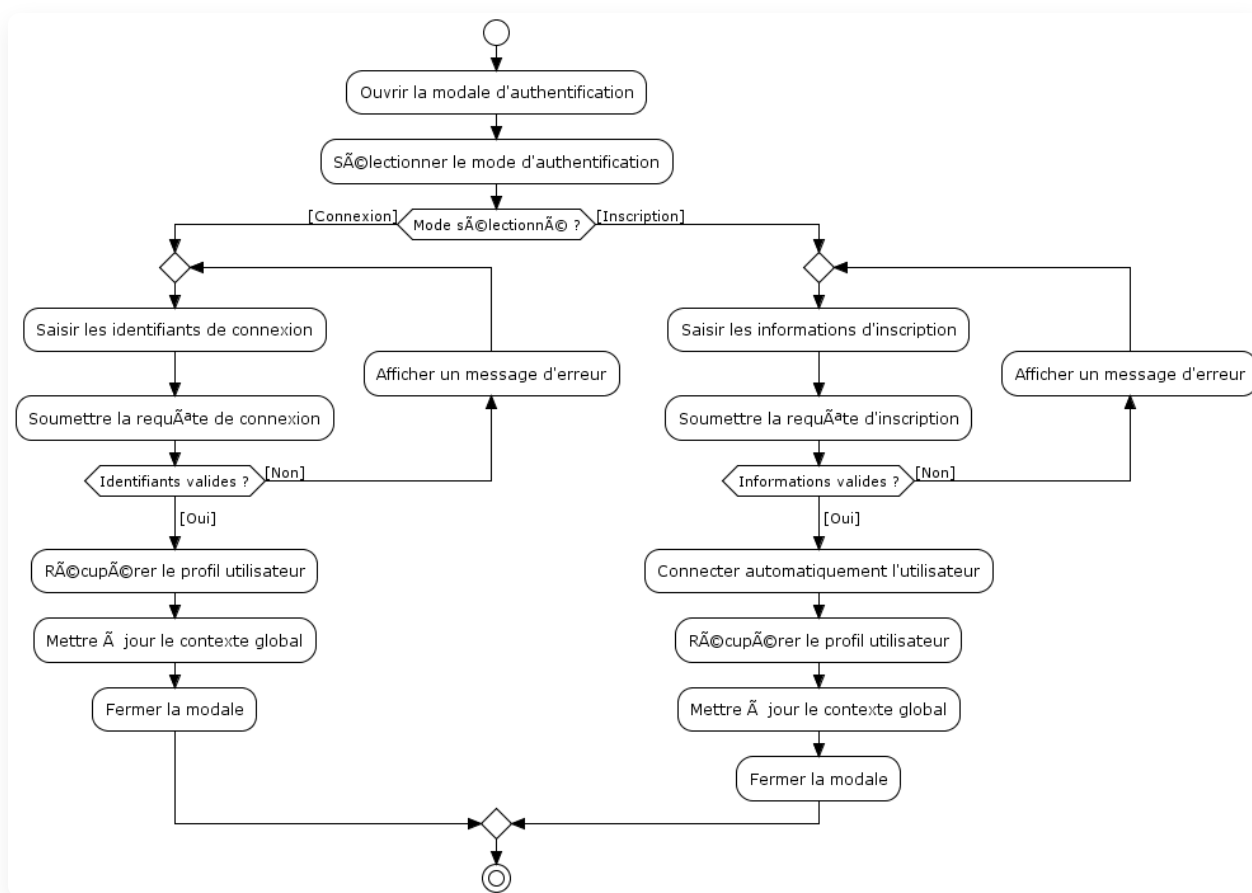
E.2.2 Processus de Recherche et de Filtrage de Films (act2.wsd)

Processus Frontend : Recherche de Films (MoviesPage.tsx)

E.2.3 Processus d'Accueil et Modale de Temps Disponible (act3.wsd)

Processus Frontend : Arrivée sur le site (Home.tsx)

E.2.4 Processus de Consentement RGPD et Authentification (act4.wsd)



E.3 Diagramme de Classes Structurel (class_diagram.wsd)

